Programmation OpenGL partie 02

Shaders : Programmation et Compilation

M. Malek Bengougam
malek.bengougam@gmail.com

Programmation OpenGL Moderne

Initiation au GLSL/ESSL

GLSL ?

- Pour Open**GL S**hading **L**anguage (ou GLslang).
- Développé par 3D Labs. Repris par l'ARB.
- Inspiré de RenderMan (Pixar) et de l'HLSL.
- Langage compilé. Syntaxe proche du C.
- Concurrents directs: CG de Nvidia et surtout HLSL / Direct3D.
- **ESSL** = OpenGL-**ES S**hading **L**anguage

Versions du GLSL

GLSL	OpenGL	Notes
110	2.0	Le mot clé « precision » n'est pas reconnu
120	2.1	Quelques ajouts mineurs pour être totalement SM 3.0
130	3.0	Nouvelle syntaxe (non encore définitive)
140	3.1	Ajout Geometry Shaders. Convergence avec GLSL ES 1.0
150	3.2	Changement de syntaxe standardisé
330	3.3	A partir d'OpenGL 3.3 le GLSL suis le numéro de version GL
400-450	4.0-4.5	Tesselation Shaders (Control et Evaluation) + Compute

ESSL	OpenGL ES	Notes
100	2.0	La majeure partie des GPU embarqués + WebGL
3x0	3.x	totalement compatible GLSL 330 (ES30) et 430 (ES31)

Le préprocesseur

- Fonctionne comme en C: #if, #define, #ifdef …
- Prenez l'habitude de spécifier le numéro de version du GLSL:
  #version <numéro_de_version_glsl_sans_le_point>

  #version 100 // au minimum pour nous
- Macros prédéfinies
  __FILE__
  __LINE__
  __VERSION__
- **GL_ES**, Permet de tester si le compilateur est celui de GLSL ES. Ex:
  #if **GL_ES**
  …
  #endif

Programmation structurée

- Sélection: **if** () {}, **if** () {} **else** {}

- Saut: **break, continue, return**

- Itération:
  for (;;) { *break, continue* }
  while () { *break, continue* }
  do { *break, continue* } **while** ();

- **discard** : annule le traitement d'un fragment (fragment shader seulement)

La fonction main()

- Il ne peut y’avoir qu’une seule fonction *main* pour un type de shader (un main VS, un main FS).
- Toujours le même prototype: **void main**(**void**).
- Les appels de fonctions sont résolus au moment du linkage des shaders en un seul programme (*shader program*) il est donc possible d’avoir des fonctions dans des shaders différents.
- Plus de détail dans le chapitre consacré à l’API Shaders.

Types de base (1)

- **void**, utilisé pour une fonction sans arguments, ou valeur de retour
- **bool**
- **int**, **uint** pour « unsigned int »
- **float**
- Ainsi que des tableaux et des structures similaires au C

- Note: par défaut les types int et float sont sur 32 bits en GLSL mais ce n'est pas forcément le cas en GLSL-ES 1.0.
- De plus la précision de ces types n'est pas toujours la même dans un vertex shader (généralement haute précision) et dans un fragment shader (généralement précision moyenne).
  - Il est possible de savoir si l'implémentation supporte un type sur 32 bits en testant la macro **GL_FRAGMENT_PRECISION_HIGH**.

Précision des types

- Il est possible de forcer la précision d'une variable à l'aide de l'un de ces 3 qualificatifs:

  highp, précision sur 32 bits, par défaut dans un vertex shader, optionnelle dans un fragment shader. Exemple:

  highp mat4 matViewProj;

  mediump, domaine standard dans un fragment shader. Exemple:

  varying mediump vec2 texCoord;

  lowp, précision et domaine faible mais suffisant pour certaines données telles les couleurs. Exemple:

  lowp float color;

- De plus il est possible de forcer la précision d'un type particulier à l'aide du mot clé '**precision**', exemple:

  precision highp float; // force globalement les float et aussi vec2/3/4, mat2/3/4

Types de base (2)

- Les vecteurs sont des structures composés de 2, 3 ou 4 éléments (entier ou réel).

- En pratique les registres du GPU sont tous de type SIMD sur 128 bits et peuvent donc contenir totalement un vecteur de 4 éléments sur 32 bits chacun.

- Les opérations sur chaque élément se font de manière indépendante, et en parallèle !

  vec2, vec3, vec4 : vecteur de 2,3 et 4 réels (***float***)
  ivec2, ivec3, ivec4 : vecteur de 2, 3 et 4 entiers (***int***)
  bvec2, bvec3, bvec4 : vecteur de 2, 3 et 4 booléens (***bool***)

- Utilisez de préférences le type qui correspond le mieux aux données (évitez les types trop large).

Constructeurs de vecteur

- En GLSL il n'est pas possible de convertir (cast) d'un type vers un autre, seul l'usage des constructeurs est autorisé:
- <type> nom = <type>(<parametres>);

```
vec3 v1 = vec3(0.0);
float s1 = float(v1);        // ou v1.x, v1.s, v1.r
vec4 v2 = vec4(v1, s1);
vec2 v3 = v2.zw;
vec4 v4 = vec4(vec3(v3, 0.0), 1.0);
```

Composantes des vecteurs

- Accès aux champs, telle une structure, à l'aide de l'opérateur '**.**' ou de manière indexée avec '**[]**'

- Il existe 3 variantes de syntaxe distinctes:

  {**x**, **y**, **z**, **w**}, points ou normales

  {**r**, **g**, **b**, **a**}, généralement pour les couleurs

  {**s**, **t**, **p**, **q**}, pour les coordonnées de texture

- Ces variantes ne sont pas mélangeable !

Swizzling et réplication

- « To swizzle » mixer, mélanger (un cocktail par exemple)

- Ici, il s'agit de mélanger les champs des vecteurs de façon combinatoire:
  vec4 colorRGBA = (…);
  vec2 colorBG = colorRGBA.***bg***;
  vec4 quaternion = (…);
  vec3 axe = quaternion.***xyz***;
  float demiAngle = quaternion.***w***;

- Y compris en réplicant un champs à plusieurs reprises (*splatting*), exemple:
  vec4 alphaVec = colorRGBA.***aaaa***;

- On peut ainsi mélanger et répéter des champs comme dans cet exemple:
  vec2 texCoords = (…);
  vec4 nexus = texCoords.***stts***;

Matrices

- Matrices 2x2, 3x3, 4x4 en réel (float) seulement.
  mat2, **mat3**, **mat4**

- En pratique, ce sont des tableaux de vecteurs colonnes où chaque colonne correspond à un registre:

```
vec2 a = vec2(1.0, 2.0);
vec2 b = vec2(3.0, 4.0);
mat2 m = mat2(a, b);
```

- On accède à chacune des colonnes à l'aide de l'opérateur '**[]**' et aux composantes des vecteurs colonnes avec '**[][]**'. Exemple:

```
mat4 transform;
transform[3] = vec4(0.0); // ou vec4(vec3(0.0), 1.0)
transform[3][3] = 1.0;
```

Structures

- Les structures sont déclarées comme en C…
- …mais avec un constructeur explicite:

```
struct directional {
    vec3 direction;
    vec3 color;
};
directional light1;
directional light2(vec3(1.0), vec3(0.8,0.6,0.4));
```

- Il est possible de déclarer des tableaux de structures, des structures de tableaux etc…

Textures

- **sampler2D**, un type pour accéder (sampler) au contenu d'une texture 2D
- **samplerCube**, un type pour accéder à une « cubemap » composée de 6 textures 2D
- (ainsi que **sampler1D**, **sampler3D,** etc…)

- Il est nécessaire d'utiliser un jeu de fonctions particulières du GLSL afin de pouvoir les sampler.
- Ces fonctions nous permettent de lire le contenu de texture en les adressant un peu comme un tableau en utilisant des coordonnées de texture.

Visibilité des variables

- Par défaut, les variables ont la même visibilité/portée qu'en C. Les variables peuvent être locales à une fonction ou à un shader.

- Il est possible de modifier le comportement d'une variable à l'aide de l'un de ces mot clé:

const, constante, en lecture seule, locale au shader.

attribute, par vertex, en lecture seule. Fait le lien entre une variable et les données de vertices qu'OpenGL envoi au GPU. Vertex shader seulement.

uniform, par primitive, en lecture seule. Le contenu de la variable provient du programme OpenGL et reste constant pendant toute l'exécution du shader.

varying, (<u>type de base réel seulement</u>) par fragment, le contenu initial de la variable provient du vertex shader et est interpolé à chaque fragment. Note: En lecture seule dans le fragment shader, lecture/écriture dans le vertex shader.

Du Vertex Shader au Fragment Shader

- Pour communiquer entre les vertex shader et les fragment shader on utilisera **varying**.
- OpenGL prédéfini des variables spéciales au niveau des vertex shader (en écriture seule) dont seule gl_Position est obligatoire:

  1. La position du vertex en clip space (NDC), **<u>à définir obligatoirement</u>**
  highp vec4 **gl_Position**;
  2. La taille d'un point en pixels (GL_POINTS)
  mediump float **gl_PointSize**;

- Note: OpenGL 2.0/2.1 prédéfini de nombreux attributs, varying ou uniform aussi bien pour les vertex shaders et les fragment shaders. <u>Nous ne les utiliserons pas</u> pour des questions de compatibilité avec OpenGL ES et OpenGL 3/4!

Fragment Shader

- Si vous avez déclaré une variable varying dans le vertex shader il faut la déclarer dans le fragment shader (et de manière totalement similaire) afin que le GLSL puisse l'utiliser.

- Il existe aussi des variables spéciales en entrée:

```
mediump vec4 gl_FragCoord;      // Position dans le frame buffer
bool gl_FrontFacing;            // le fragment appartient-il à une front face
mediump int gl_PointCoord;      // (GL_POINTS seulement)
```

- En sortie, l'une de ces deux variables est obligatoire:

```
mediump vec4 gl_FragColor;      // couleur RGBA du fragment
mediump vec4 gl_FragData[n];    // couleur RGBA du fragment sur le point
                                d'attachement numéro 'n' (cf. chap. 08-FBO)
```

Fonctions fondamentales (1)

- http://www.khronos.org/opengles/sdk/docs/reference_cards/OpenGL-ES-2_0-Reference-card.pdf

- **Angles et Trigonometrie**

  radians(), degrees(), Sin(), cos(), tan(), asin(), acos(), atan()

- **Geometrie**

  length(), distance(), dot(), cross(), normalize(), faceforward(), reflect(), refract()

- **Comparaison**

  any(), all(), not(), equal(), notEqual(), lessThan(), lessThanEqual(), greaterThan(), greaterThanEqual()

Fonctions fondamentales (2)

- **Numériques**:

  abs(), sign(), min(), max(), floor(), ceil(), fract(), mod(), clamp(), min(), step(), smoothstep()

- **Exponentielles**:

  pow(), exp(), log(), exp2(), log2(), sqrt(), inversesqrt()

- **Accès aux textures**:

  texture2D(), texture2DProj(), textureCube()

Extensions du GLSL

- Tout comme l'API OpenGL il est possible d'utiliser des extensions spécifiques à une version du GLSL ou d'un pilote.
- On utilise la directive du préprocesseur **#extension** suivie du nom de l'extension et d'un paramètre d'activation, exemple:

#extension GL_ARB_geometry_shader4 : **enable**

- Le paramètre peut être '**enable**', '**require**', '**disable**', '**warn**'
- On peut ensuite tester la présence de l'extension à l'aide du préprocesseur:

```
#ifndef GL_ARB_geometry_shader4
#error ''GL_ARB_geometry_shader4 requis!'';
#endif
```

API Shaders
et
compilation des shaders

Rappel: « Objets » OpenGL

- OpenGL fonctionne en interne en allouant un tableau pour chaque objets qu'il doit gérer.

- A chaque fois que l'on a besoin d'un objet, OpenGL nous renvoi une valeur de type **GLint** ou **GLuint** (appelé « *handle* », ou « id », pour « identifiant ») qui nous permettra de référencer et manipuler facilement ces objets OpenGL.

- Lorsque le type retourné est **GLuint**, une valeur de '**0**' indique que le *handle* est invalide.
- Mais si le type de retour est **Glint**, c'est la valeur '**-1**' qui indique que le *handle* est invalide.

  – On utilisera dans ce cas **glGetError**() afin d'obtenir le code erreur OpenGL indiquant l'origine de l'erreur.

Shader Object

- En premier lieu il nous faut créer un objet qui va contenir chacun de nos shaders à l'aide de la fonction:

GLuint glCreateShader(GLenum type)

- Pour détruire le shader object on utilisera:

void **glDeleteShader**(**GLuint** shaderObject)

- Plusieurs types de shaders existent mais seuls ceux-ci sont supportés en OpenGL ES 2.0

GL_VERTEX_SHADER, GL_FRAGMENT_SHADER

Spécifier le code source

- Une fois le Shader Object créé il faut fournir le code source du shader sous forme d'une ou plusieurs chaines de caractères via la fonction suivante:

void **glShaderSource**(**GLuint** shaderObject, **GLsizei** count, **const char** ****shaderString, **const GLint** ***shaderLength)

- Remarquez que la fonction prend comme deux derniers paramètres deux tableaux: un tableau de chaîne de caractères représentant le code de chaque shaders, et un tableau indiquant la taille de chaque string.
- On spécifie le nombre d'éléments dans les tableaux à l'aide du second paramètre 'count'.
- Si les chaines de caractères sont terminé par le caractère spécial '**\0**' (null) on peut se contenter de spécifier **NULL** pour le dernier paramètre.

Note: certaines implémentations donne la possibilité de fournir du code précompilé. Cela peut-être intéressant sur une architecture fermée type console ou embarquée, mais pas sur PC/Mac où les GPU sont très divers.

Compilation

- Pour compiler le shader object on utilisera simplement la fonction :

  void **glCompileShader**(**GLuint** shaderObject)

- Il est possible (entre autre) de vérifier le résultat de la compilation à l'aide de:

  glGetShaderiv(**GLuint** shaderObject, **GLenum** pname, **GLint** *params)

- Cette fonction permet d'obtenir des informations sur un Shader Object. Par exemple avec pname = **GL_COMPILE_STATUS** on obtient en retour **GL_TRUE** ou **GL_FALSE** selon que la compilation ait réussi ou non.

Information et diagnostic

- Pour des informations ou diagnostic sur un Shader Object en cas d'erreur de compilation:

  glGetShaderInfoLog(**Gluint** shaderObj, **GLsizei** bufferSize, **GLsizei** ***length**, **char** ***log**)

- En entrée il faut fournir la taille du buffer préalablement alloué (avec 1 octet de moins pour le 0 final de la chaine de caractère) et dont on passera l'adresse en dernier paramètre. 'length' indique pour sa part le nombre de caractères réellement écris dans le buffer.

  Note: La taille du texte du diagnostic peut s'obtenir avec les fonctions glGetShaderiv() et GL_INFO_LOG_LENGTH

Shader Program

- Un Shader Program permet de faire le lien entre plusieurs shaders (de tout type) et notamment de déterminer quelles sont les variables communiquées de l'API vers le shader, et d'un shader vers un autre.

- Tel un compilateur C++, ceci permet d'optimiser l'utilisation des registres (ou « location » GLSL).

- Création/destruction du Shader Program:

```
GLuint glCreateProgram()
void glDeleteProgram(GLuint shaderProgram)
```

Spécifier les Shader Object à lier

- Une fois le Shader Program créé, il faut indiquer quels sont les Shader Objects à lier (attacher) ou délier (détacher) au programme:

void **glAttachShader**(**GLuint** shaderProgram, **GLuint** shaderObject)

void **glDetachShader**(**GLuint** shaderProgram, **GLuint** shaderObject)

Note: il est possible, pour libérer de la mémoire par exemple, de détacher et détruire un Shader Object une fois que le Shader Program a été correctement lié.

Linkage

- Le lien effectif se fait à l'aide de la fonction

  void **glLinkProgram**(**GLuint** shaderProgram)

- De la même manière que pour la compilation, il est possible de vérifier le statut du link avec **GL_LINK_STATUS**

  void **glGetProgramiv**(**GLuint** shaderProgram, **GLenum** pname, **GLint** *params)

- Finalement, si le lien n'a pas rencontré de problème il ne nous reste plus qu'à valider le shader pour être sûr qu'il tourne sans problème sur ce GPU:

  void **glValidateProgram**(**GLuint** shaderProgram)

Information et diagnostic

- Pour des informations ou diagnostic sur un Program Object en cas d'erreur de lien ou de validation:

  void **glGetProgramInfoLog**(**GLuint** shaderProgram, **GLsizei** bufferSize, **GLsizei** *length, **char** *log)

- En entrée il faut fournir la taille du buffer préalablement alloué - 1 octet, et dont on passera l'adresse en dernier paramètre. 'length' indique pour sa part le nombre de caractères écris dans le buffer.

  Note: La taille du texte du diagnostic peut s'obtenir avec les fonctions **glGetProgramiv**() et **GL_INFO_LOG_LENGTH**

Utilisation d’un Shader Program

- Finalement, pour que les shaders soit utilisé lors du rendu il faut les définir comme actif.

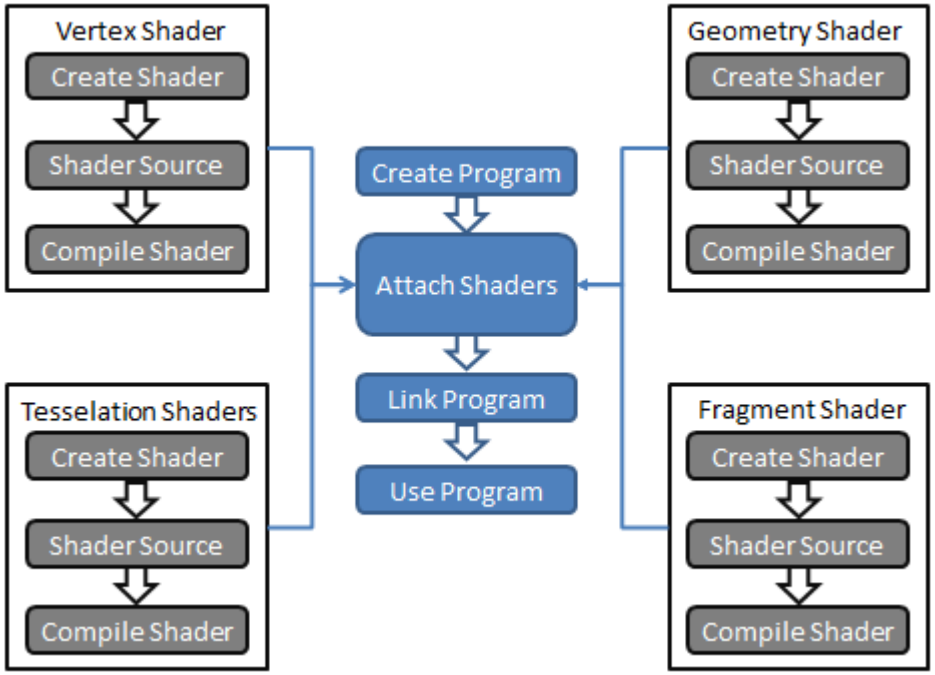
- Un seul Shader Program peut être actif à la fois. Pour activer un shader program:

```
void glUseProgram(GLuint shaderProgram)
```

- Pour désactiver tout Shader Program il suffit de passer ‘**0**’ en paramètre:

```
glUseProgram(0);
```

Récapitulatif



- http://www.lighthouse3d.com/tutorials/glsl-core-tutorial/setup/

Communication avec le GLSL

Attributs de vertex

- Par défaut OpenGL affecte automatiquement un registre (location) à un attribut. La fonction suivante permet de récupérer l'identifiant du registre:

  GLint **glGetAttribLocation**(**GLuint** shaderProgram, **const char** ***name)

- Alternativement, il est possible de spécifier soi-même le numéro d'identifiant. Ceci doit être fait AVANT la phase de lien (link) afin qu'OpenGL puisse le prendre en compte.

  void **glBindAttribLocation**(**GLuint** shaderProgram, **GLuint** index, **const char** ***name)

Variables uniformes

- OpenGL gère lui-même l'emplacement des « uniform ». L'identifiant du registre se récupère avec:

  GLint glGetUniformLocation(**GLuint** shaderProgram, **const char** *name)

- Et les fonctions suivantes permettent d'affecter une valeur à n'importe quelle variable uniforme du Shader Program.

  void glUniform{1234}{if}(**GLint** location, T value)

  void glUniform{1234}{if}v(**GLint** location, Glsizei count, T* value)

  void glUniformMatrix{234}fv(**GLint** location, **Glsizei** count, **GLboolean** transpose, **const** float *value)

Autres fonctions utiles

- On peut aussi obtenir des informations sur un attribut avec

  glGetActiveAttrib()
  glGetVertexAttribfv()
  glGetVertexAttribiv()

- Et pour obtenir des informations sur une variable *uniform*

  glGetActiveUniform()
  glGetUniformfv()
  glGetUniformiv()

Communication inter shader

- La seule façon de communiquer des données d'un shader vers un autre :

  Spécifier la variable comme étant **varying**

- Les données ne peuvent être envoyées que dans le sens du pipeline, c'est-à-dire:

  VS -> **FS**